

Over the past few weeks we learned how AI systems get built:

1. Fine-tuning (control format, tone)
2. Real-time retrieval (RAG) → (company-specific, use-case-specific knowledge)
3. Agentic systems (tool-use, MCP, orchestration)

Once these systems are built and deployed, they are surprisingly hard to improve.

1. Users report the agent "feels worse" after a change
2. The team has no way to verify except to guess-and-check by hand.
3. Absent evals, debugging is reactive – you only find out it broke when someone complains.

The deeper pain:

Engineers can't distinguish a real regression from random noise

Can't automatically test a change against hundreds of scenarios, and can't **measure** whether they actually improved anything.

LLMs are **non-deterministic** – the same prompt can give different answers.

So "it looked fine when I tried it once" is not evidence. That's exactly the gap evals fill.

The agenda

1. What are Evals? → tests for AI systems
2. Human-guided Evals → start manual: dogfooding + intuition
3. Automated Evals → LLM-as-a-judge, at scale
4. Monitoring & Observability → watch it in production

What are Evals?

Evals are tests for AI systems. They check the **behavior** and the **quality** of an AI system's outputs.

but unlike a normal unit test (`assert add(2,2) == 4``), the output is fuzzy text, so we need fuzzy-but-rigorous ways to grade it.

Evals for plain LLMs

For a raw LLM (no tools, no retrieval), evals test general capabilities: **relevance, reasoning, and math.**

The industry runs these as standardized **benchmarks**:

MMLU → Massive Multitask Language Understanding (broad knowledge, 57 subjects)

GPQA → Graduate-level, Google-Proof Q&A (hard science, expert-level)

HELM → Holistic Evaluation of Language Models (a whole battery of metrics)

These are how you see headlines like *"Model X scores 88% on MMLU."*

They grade the *model*

Evals for RAG systems

RAG pipeline

```
query → [ RETRIEVAL ] → chunks → [ LLM + prompt ] → answer
      |                   |
      test this          test this
```

(1) Retrieval quality – did we fetch the right chunks?

Metrics: Precision, Recall, Relevance

Knobs you turn: chunking strategy, embedding model

(2) Answer quality – given good chunks, did we write a good answer?

Metrics: Groundedness, Answer relevancy, Correctness

Knob you turn: the system prompt

Why split them?

If the final answer is bad, you need to know *whose fault it is*.

Bad retrieval (wrong chunks) and bad generation (good chunks, lazy answer) are fixed in completely different places.

Measuring them separately tells you where to spend your time.

The retrieval metrics

"Precision, Recall, Relevance"

****First, you need a golden dataset.**** None of these metrics work without ****ground truth****: a human (your SME) labels, for each query, **which** chunks are actually relevant. That labeled set is your answer key.

```

...
    Query: "What is the refund window?"
    Golden truth (relevant chunk IDs): { C2, C5, C9 }      ←
there are 3 right chunks
    Your retriever returns top-5:      [ C2, C7, C5, C1, C8 ]
                                     ✓   ×   ✓   ×   ×
...

```

Precision@K – "of what I fetched, how much was useful?" (junk control)

```

...
    Precision@5 = (relevant in top-5) / K = 2 / 5 = 0.40
...

```

Use it when: your LLM gets distracted by noise. Low precision = you're stuffing the prompt with irrelevant chunks.

Recall@K – "of all the right chunks, how many did I catch?" (miss control)

```

...
Recall@5 = (relevant retrieved) / (total relevant) = 2 / 3 =
0.67      ← missed C9
...

```

Use it when: answers are **incomplete**. Low recall = the needed info never reached the LLM.

<u>Symptom</u>	<u>Likely metric</u>	<u>Fix (the knob)</u>
"answers feel incomplete"	low Recall@K	K, smaller chunks, better embeddings
>"answers wander off-topic"	low Precision@K	↓ K, add a re-ranker

Evals for agentic systems – outcome, process, safety

Agents **do things**, so we check three layers:

1. Outcome → did it succeed? task success, pass@k on a benchmark

2. Process → did it work right? tool-call traces (accuracy, ordering, selection)

3. Safety → did it stay safe? tool-call traces: hallucinations, toxicity, guardrail adherence

> A system is evaluated on the parameters that are measurable *and* changeable in it.

Concretely:

You don't evaluate what you can't change.

Pick metrics that point at a ****knob** you can actually turn.**

...

A RAG-based system → CAN be checked for answer relevance
 - → CANNOT be checked for tool-calling order
 (it has no tools)

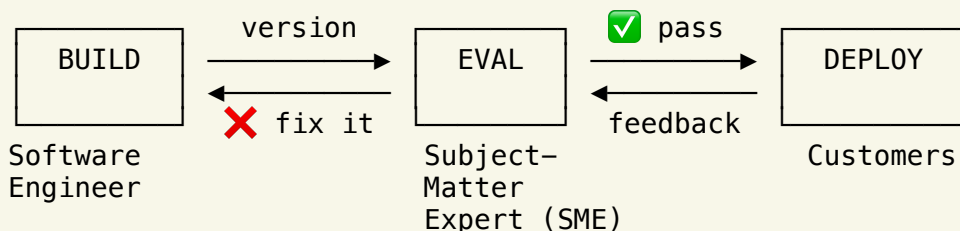
... An agent → CAN be checked for tool-calling order
 ...

Human-Guided Evals

When building agents, ****start with manual testing, dogfooding, and intuition.**** **(Dogfooding = using your own product like a real user before anyone else does.)**

The Build → Eval → Deploy loop

The lifecycle of shipping an agent looks like this



1. SME creates 20–50 user queries that are important to get right.
2. Engineer runs the queries as input to the agent, and notes the response.
3. Engineer changes the system prompt (preferably with the SME's guidance).
4. The system is evaluated again on the SAME prompts.
5. Continue until it's deployable.

>Example – a support bot.

> Query : "How do I get a refund after 30 days?"
> v1 answer: "You can't, sorry." ←
SME: too blunt, and WRONG

> Prompt change: add policy doc + "cite the exception clause"

> v2 answer: "After 30 days refunds need manager approval – here's how to request it..." ←

SME:  correct & kind

> Same query, measurable improvement. Repeat across all 50.

When an answer is bad, don't just change the prompt randomly.
Localize the failure:

```

...
What is the root cause?
├─ Retrieval → vector DB / chunking(wrong chunks fetched?)
├─ Tool discovery → tool name & description (LLM didn't know the tool fit?)
└─ Tool calling → arguments, response type, (called it wrong / in wrong order?)order of tool use
...
```

Each root cause has a different fix – you don't rewrite the prompt when the real problem is a bad chunking size.

Once real users arrive, study `user\_query\_patterns`.

- You can't build effective database indexes without knowing the data-access patterns.
- Similarly, you can't build effective evals without knowing the user-query patterns.

Two patterns

...

What kind of queries do users usually ask? → Frequency-based

What queries/traces dissatisfy users most? → Feedback-based

...

Your eval set should be dominated by the high-frequency queries (cover the common case) and the high-pain queries (the ones generating complaints).

Benefits of Human Evaluation

- Covers the most important queries in the app
- Builds domain-specific response patterns the LLM should follow

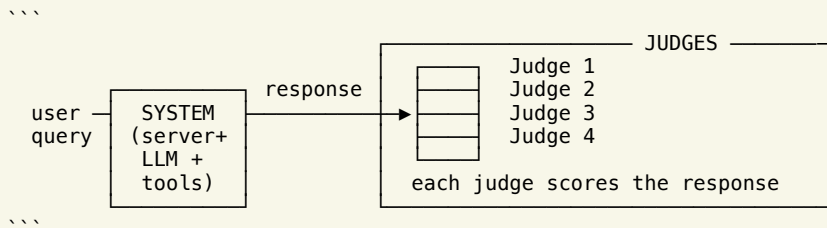
Drawbacks

- ① Not scalable
- ② Expensive – needs real SME time (their hours are costly)

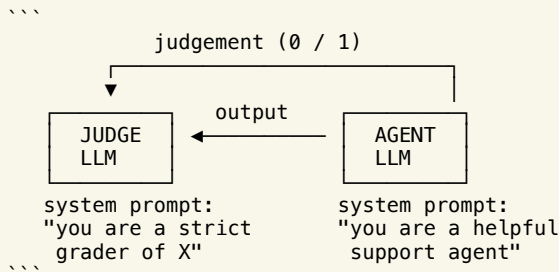
Automated Evals – LLM as a Judge

What if we use an LLM – or a panel of LLMs – to judge our system's responses?

Instead of an SME grading every answer, an LLM grader (a "judge") scores them at machine scale:



>> It *looks* circular – an LLM checking an LLM.



> It seems like a cyclic dependency, but since the system prompts (and therefore the goals) of the two LLMs are different, we can assume one LLM can judge the other. The Judge can be a more powerful model than the agent LLM

The judging metrics

Generation quality (is the answer itself good?)

```

1. Answer relevancy → on-topic and to-the-point
2. Groundedness . → a.k.a. Faithfulness: claims are backed by the source material  
(Faithfulness) (no hallucinations)
3. Coherence → not verbose; logical flow of thought
4. Completeness → no missing information
5. Topic-adherence → sticks to its topic even when the user asks irrelevant things

```

Retrieval quality (did we feed it the right context?)

```

6. Context relevance → checks retrieval quality – were the fetched chunks useful?
7. Noise sensitivity → add irrelevant docs to the context on purpose, then check  
how robust the generation stays (does junk

```

Groundedness vs. Relevancy – don't confuse them.

Relevancy asks "does the answer address the question?"

Groundedness asks "is the answer actually supported by the retrieved documents?"

Judging styles

1. Point-wise → score ONE output on its own. FAST.
(e.g. "rate this answer 0 or 1")
2. Pair-wise → compare TWO and pick the better. SLOWER.
(great for "is the new version better than the old one?")
3. List-wise → rank-order N outputs. SLOW.
(used to check re-ranker performance)

Rule of thumb:

Point-wise for cheap pass/fail at scale,

Pair-wise for regression checks (v2 vs v1),

List-wise when you're tuning a ranking/retrieval component.

Best practices for LLM-as-a-judge

...

- ☑ Each judge does ONE thing only → Scoped
- ☑ Responses are binary (0/1) → Formal
- ☑ Bias has been considered → Quality

...

1. Scoped → one judge per metric.

2. Formal– Responses should be binary 0/1 (or a tight scale 0–5).
Vague "rate 1–10" answers drift; binary is reproducible and easy to aggregate.

3. Quality (mind the biases) – LLM judges have known, documented biases:

- Long-response bias → judges favor longer answers.
Fix: cap/normalize response size.
- First-response bias → in pair-wise, judges favor whatever came first.
Fix: evaluate each pair in BOTH directions (A,B and B,A).
- Self-preference bias → a model rates ITS OWN outputs higher.
Fix: obfuscate which model wrote it; use a "jury" of different models instead of one judge.

...

judges return verdicts



Manual:	fix the prompts the judges flagged
Automated:	let the system rewrite & re-test its own prompts – "hill-climbing" (e.g. AlphaEvolve)

...

Note** – never trust a single run

Remember the whole reason we started: "*We can't tell a real regression from noise.*"

LLMs are non-deterministic, so one run proves nothing.

Run each query N times (e.g. 5–10), score every run, and compare the **averages** – not single outputs.

```

Query: "What's the refund window?" (run 5× per version)

---

v1 relevancy scores: [1, 0, 1, 1, ~~0~~] → average 0.60

v2 relevancy scores: [1, 1, 1, 1, 1] → average 1.00

▲ a REAL

improvement, not luck

```

If v1 had scored `[1,1,1,0,1] = 0.80` and v2 `[1,0,1,1,1] = 0.80`, the "improvement" you **thought** you saw in one lucky run would vanish – that's noise, correctly caught.

Averaging across runs (and across your 50–query set) is what turns "feels better" into "***measurably*** better."

The Tooling Landscape

We don't build all this from scratch

Job	Industry-standard tools
RAG metrics (out of the box)	RAGAS(precision@k, recall@k, groundedness)
LLM-as-a-judge (write & run judges)	DeepEval, TruLens, Promptfoo
Tracing + eval platform	LangSmith, Braintrust, Arize Phoenix (log runs, score, dashboards)

Example of how they fit together:

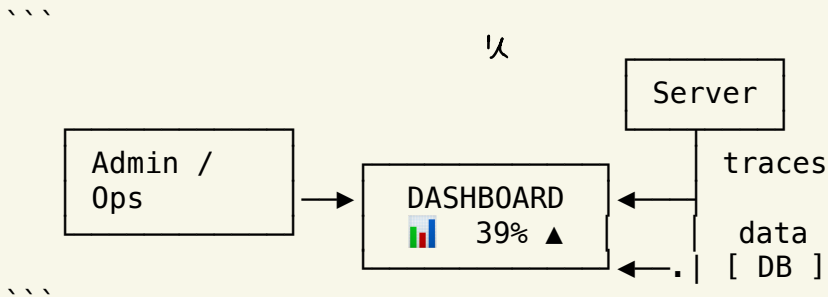
1. Wrap your RAG app with LangSmith → every query/response/trace is logged.
2. Point RAGAS at those traces → auto-computes Recall@K, Precision@K, Groundedness on your golden set.
3. Add DeepEval judges in CI → on every pull request, re-run the 50-query set; block the merge if avg score drops.

That third step is the payoff: **evals become a regression gate.**

Just like unit tests block a bad code merge, an eval drop blocks a bad **prompt/model** merge – automatically.

Monitoring & Observability

Traditional distributed systems use **logging, tracing, and aggregates** to monitor the overall health of a system.



These dashboards watch the classic four:

cost • latency • error rates • server load

AI-native systems must add

Classic metrics tell you the system is *up* – not that it's *good*.

AI systems degrade silently: nothing throws a 500, the answers just quietly get worse.

So in addition to the four above, ~~monitor~~:

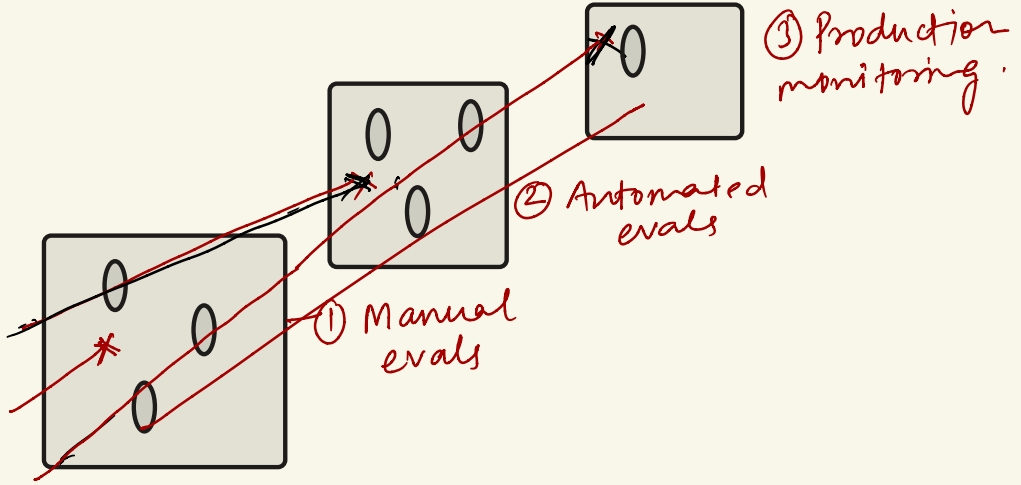
1. User feedback → reports, 👍/👎 scores (are real users unhappy?)
2. Judge scores → run your LLM judges on live traffic, and watch for DRIFT – a slow slide in scores over time

The Swiss-Cheese Model

No single layer of evals is perfect. **Each** has holes.

But stack them, and a failure has to slip through *every* layer's holes to reach the user.

That's the Swiss-cheese model of defense:



\\

- ① Manual Evals → high quality, catches subtle/domain errors, low volume
- ② Automated Evals → high volume, catches regressions at scale
- ③ Production Monitor → catches what only shows up with real users + drift

\\